

## **Ontwerpverantwoording**

Mijn oplossing is opgesplitst in drie lagen : de presentatielaag, de domeinlaag et de data laag. In de presentatielaag vinden we de verschillende pagina's van de app, de viewmodels en de navigatieservice.

Elke pagina is gebind aan een viewmodel. Zo zijn de UI en de logica strict gescheiden, en de UI heeft exact wat ze nodig heeft. Bv. het model van het domein kan een geboortedatum bevatten maar de viewmodel bevat een Leeftijd property. Zo heeft de UI exact wat ze nodig hebt en we moeten niks aanpassen aan de domein klassen. Viewmodels worden ook gebruikt bij CollectionViews zowel bij de takenlijst als de personenlijst. Zo heeft elke item van de collectie exact wat het nodig hebt.

De presentatielaag bevat ook een build-in DI-Container. Met die kunnen we het 'Inversion of Control' principe via dependency injection automatisch toepassen in onze app. (Als MAUI een pagina opbouwt, er wordt naar de parameters gekeken, en er wordt exact wat nodig is naar de pagina's gegeven.) Het zijn niet de klasse zelf die maakt de objecten die ze nodig hebt voor zijn dependencies, maar die krijgen ze dependencies van buitenaf, via de parameters van zijn constructor. De pagina's hebben viewmodels nodig, de viewmodels hebben zelf services nodig, en de services hebben zelf repositories nodig. We registreren ze allemaal in de MauiProgram.cs en die worden automatisch gegeven wanner nodig.

De domeinlaag bevat de models klassen, mappers, factories, filter/sorteerstrategieën, interfaces, services, message klassen en DTOs. De twee domein klassen zijn Taak en Persoon, een persoon wordt gelinkt aan een Taak met de PersoonId property in de Taak klasse.

In de services gebruik ik WeakReferenceMessenger. Door dit mechanisme toe te passen, als er iets wordt aangepast in de data in de enige pagina, (dan gaat het door een service) wordt er in de service een bericht gestuurd, en zo kunnen andere pagina's die ook die data gebruiken op de hoogte zijn van de data die net aangepast werd.

In de domeinlaag kunnen we ook verschillende filter/sorteerstrategieën terugvinden. Ik gebruik de strategy pattern om in de UI taken te kunnen filteren/sorteren met verschillende strategieën. De takenlijstviewmodel heeft een property HuidigeStrategie van de type IFilterStrategie. Ik maak gebruik van een interface zodat HuidigeStrategie kan verschillende strategieën bevatten, op het conditie dat ze een methode 'Filter' bevatten. Zo hebben we in onze viewmodel geen grote if/switch structuur die we moeten aanpassen bij elke nieuwe strategie. Als we later een nieuwe strategie willen toevoegen, moeten we gewoon een nieuwe klasse maken, en

nieuwe button maken in onze UI en die linken aan een nieuwe Command en dat is het. We moeten niets veranderen aan een enum, of de RefreshTakenlijst methode aanpassen.

In mijn app gebruik ik ook de factory pattern. Alle Taak of Persoon objecten die worden aangemaakt of geupdated worden gemaakt/geupdated in die factories. Zo zijn de CreatieDatum en GewijzigdDatum logica op één enkele plaats gecentraliseerd, in een factory. Als later het maken of updaten van onze Taak of Persoon objecten moet veranderen, we moeten aan één enkele plaats aanpassing doen.

In de data laag vinden we 2 repositories, de TaakRepo en de PersoonRepo. In mijn app gebruik ik de repository pattern. De dataopslag logica wordt geencapsuleerd in aparte klassen. Dit heeft het voordeel dat als de er veranderingen moeten gebeuren kwa dataopslag, de domeinregels/services enz. moeten niet aangepast worden. Zo heeft elke klasse een aparte verantwoordelijkheid. In de services van mijn app gebruik ik interfaces voor de repositories en niet rechtstreeks de concrete klasse die de dataopslag logica bevatten (hier met LiteDB). Zo zijn de services minder afhankelijk van een concrete implementatie van een repo. En als we willen later bijvoorbeeld overschakelen naar een echte SQL databank, dan moeten we gewoon een nieuwe klasse schrijven voor de SQL dataopslag, die onze interface implementeert en die in de DI-Container registreren, en dat is het. We moeten niets aanpassen in onze services.